

Day 3: Lab Exercises — Distributed Computing (MPI) & Asynchronous Programming

Duration: 4 hours (self-paced hands-on practice)

Reference: [day3/README.md](#), [day3/README-session1.md](#), and [day3/README-session2.md](#)

Note: These exercises are for personal practice to reinforce the concepts covered today. Start from Exercise 1 and work sequentially.

Exercise 1: MPI Token Ring Communication

Topics: `mpi4py`, Point-to-point communication, `comm.sendrecv()`, non-blocking `isend/irecv`

Problem Statement

Write an MPI script `ex01_mpi_ring.py` that implements a circular token-passing ring across all ranks:

Rank 0 $\longrightarrow$ Rank 1 $\longrightarrow$ Rank 2 $\longrightarrow$ Rank 3 $\longrightarrow$ Rank 0

1. Initialize MPI with `comm = MPI.COMM_WORLD`, finding your `rank` and `size`.
2. Ensure the script runs with at least 3 processes (`mpirun -n 4 python3 ex01_mpi_ring.py`).
3. Set up the ring topology:
 - Next rank: `(rank + 1) % size`
 - Previous rank: `(rank - 1 + size) % size`
4. **Token Passing:**
 - Rank 0 originates a token: `token = {"hops": 0, "history": [0]}`.
 - Rank 0 sends the token to Rank 1 and then waits to receive the final token from Rank $(N - 1)$.
 - Intermediate ranks wait to receive the token from the previous rank, increment `hops += 1`, append their rank to `history`, and forward the token to the next rank.
5. Prevent deadlocks:
 - Use `comm.sendrecv()` or non-blocking `comm.isend()` / `comm.irecv()`.
6. When the token returns to Rank 0, print the total hop count and the path traversal history.

Hints & Guidelines

- Launch your program using: `mpirun -n 4 python3 ex01_mpi_ring.py`.
- If testing on a machine without `mpi4py`, write the code using standard `mpi4py` syntax and test under simulated conditions or within an MPI-enabled container/virtual machine.

Best Practices

- Ring topologies are classic synchronization primitives in distributed algorithms.
- Always use modular arithmetic (`% size`) to cleanly wrap around boundary ranks.

Exercise 2: Distributed Monte Carlo Pi with MPI Collectives

Topics: `comm.bcast`, `comm.scatter`, `comm.reduce`, `MPI.SUM`, parallel reduction

Problem Statement

Write an MPI script `ex02_mpi_collective_pi.py` that computes π in parallel using collective operations:

1. Broadcast Configuration:

- Rank 0 specifies the total number of Monte Carlo samples (e.g., 12,000,000).
- Rank 0 broadcasts this number to all processes using `comm.bcast(total_samples, root=0)`.

2. Parallel Sampling:

- Each rank calculates its local quota: `local_samples = total_samples // size`.
- Each rank seeds its random number generator with `random.seed(rank * 100 + 42)` to ensure independent sample generation.
- Each rank generates points $(x, y) \in [0, 1] \times [0, 1]$ and counts how many satisfy $x^2 + y^2 \leq 1$.

3. Global Reduction:

- Use `comm.reduce(local_inside, op=MPI.SUM, root=0)` to sum the points inside the unit circle across all ranks onto Rank 0.

4. Result Calculation:

- On Rank 0, compute the approximation: $\pi \approx 4 \times \frac{\text{total_inside}}{\text{total_samples}}$.
- Print the calculated value, the error compared to `math.pi`, and the execution runtime.

Hints & Guidelines

- Every process in the communicator must call `comm.bcast()` and `comm.reduce()`. If any rank omits the collective call, all other processes will hang!
- Notice that no process ever sends raw random coordinates over the network—only the single aggregated scalar integer is reduced, maximizing network efficiency.

Best Practices

- Never transmit bulk raw data if the reduction can be performed locally on each node first.
- In collective communication, ensure that non-root ranks pass `root=0` as an argument.

Exercise 3: High-Level Concurrency with `ThreadPoolExecutor`

Topics: `concurrent.futures`, `ThreadPoolExecutor`, `as_completed()`, timeouts

Problem Statement

Write a script `ex03_futures_downloader.py` that fetches simulated resources concurrently:

1. Define a list of 10 simulated web endpoints with varying simulated network delays (between 0.2 and 1.5 seconds).
2. Create a worker function `download_endpoint(url)`:
 - Simulates network latency using `time.sleep()`.
 - If the URL ends with `.error`, deliberately raise a `ConnectionError`.
 - Otherwise, returns a string containing the downloaded payload.
3. Use `concurrent.futures.ThreadPoolExecutor(max_workers=5)`:
 - Submit all 10 download tasks using `executor.submit()`.
 - Map each returned Future object back to its URL using a dictionary.
4. Use `as_completed()` to iterate over results as soon as they finish:
 - Handle successful downloads and print their byte sizes.
 - Catch and log exceptions gracefully when encountering `.error` URLs without crashing the script.
 - Implement a per-task timeout using `future.result(timeout=1.0)` to abort endpoints that take too long.

Hints & Guidelines

- `as_completed(future_dict)` yields Futures in the order they complete, not in the order they were submitted.
- Accessing `future.result()` re-raises any exception that was thrown inside the worker thread.

Best Practices

- Always attach timeouts to futures when making network or external system calls.
- Use `ThreadPoolExecutor` for I/O-bound tasks and `ProcessPoolExecutor` for CPU-bound tasks with the exact same API.

Exercise 4: Cooperative Concurrency with `asyncio`

Topics: `asyncio`, `async def`, `await`, `asyncio.gather()`, event loop

Problem Statement

Write an asynchronous monitoring tool `ex04_async_monitor.py` using Python's `asyncio`:

1. Define three asynchronous coroutines simulating microservice health checks:
 - `check_database()`: Takes 0.4s (`await asyncio.sleep(0.4)`), returns `{"db": "HEALTHY", "latency_ms": 12}`.
 - `check_redis_cache()`: Takes 0.2s, returns `{"cache": "HEALTHY", "latency_ms": 4}`.

- `check_storage_service()`: Takes 0.6s, returns `{"storage": "HEALTHY", "latency_ms": 28}`.

2. Write a top-level coroutine `run_health_checks()`:

- Uses `asyncio.gather()` to launch all three health checks concurrently on the single-threaded event loop.
- Records the total time taken to complete all checks.
- Verifies that the total execution time is roughly equal to the slowest individual check (~0.6s), rather than the sum of all three ($0.4 + 0.2 + 0.6 = 1.2$ s).

3. Run the top-level coroutine using `asyncio.run()`.

Hints & Guidelines

- Never call blocking functions like `time.sleep()` inside coroutines—always use `await asyncio.sleep()`.
- Coroutine functions declared with `async def` do not run when called; they return a coroutine object that must be `awaited` or scheduled on the event loop.

Best Practices

- `asyncio` is far more scalable than threads for network I/O, capable of managing tens of thousands of active socket connections with negligible memory overhead.
- Use `asyncio.gather()` when you have a known set of concurrent tasks and need all of their results returned together.

Exercise 5: Bridging Asyncio with CPU-Heavy Workloads

Topics: `asyncio.create_task()`, task cancellation, `loop.run_in_executor()`

Problem Statement

Write a script `ex05_async_cpu_bridge.py` that demonstrates how to execute heavy CPU computation alongside an active asynchronous event loop:

1. The Problem:

- If a coroutine executes a heavy `for` loop or matrix calculation directly, it blocks the single-threaded event loop, freezing all other asynchronous tasks!

2. The Solution:

- Create a background heartbeat coroutine `heartbeat_ticker()` that prints `"[Heartbeat] Active and responsive..."` every 0.25 seconds.
- Create a heavy synchronous CPU function `fibonacci_cpu(n)` that calculates a large Fibonacci or prime number.

3. In the main `async` function:

- Start the heartbeat as a background task using `asyncio.create_task()`.

- Obtain the running event loop using `loop = asyncio.get_running_loop()`.
- Offload `fibonacci_cpu` to the default thread pool executor using `await loop.run_in_executor(None, fibonacci_cpu, 35)`.
- Observe that the heartbeat ticker **continues printing regularly** while the CPU function calculates in the background!
- Cancel the heartbeat task cleanly using `task.cancel()` once the calculation finishes.

Hints & Guidelines

- `loop.run_in_executor(None, func, *args)` runs `func` in a background `ThreadPoolExecutor` and returns an asyncio Future that can be awaited.
- Catch `asyncio.CancelledError` inside the heartbeat task to allow it to perform clean exit procedures.

Best Practices

- Never allow CPU-bound work or synchronous third-party libraries to execute on the main `asyncio` thread.
- Always cancel or await background tasks before shutting down an asyncio event loop to avoid `Task was destroyed but it is still pending` warnings.